PRACTICAL 10

Docker Compose for a Multi-Container
Web-Based AI Application

Environment: Ubuntu on WSL 2 with Docker Desktop

Services: Frontend (Nginx) • AI Model (Flask + Scikit-learn) • Database (MongoDB)

Beginner-friendly practical record: setup from scratch, every command, complete source files, build and run steps, testing, troubleshooting, architecture, and viva questions.

Follow the steps in order. Enter commands in the Ubuntu/WSL terminal unless a step says to use the browser.

1. Aim

To use Docker Compose to orchestrate and manage multiple containers for a web-based AI application, including separate containers for the AI model, database, and frontend.

2. Learning objectives

• Create a multi-service application project from scratch.

• Build a Docker image for a Flask machine-learning API.

• Run MongoDB as a separate database container.

• Serve a web interface using Nginx in a separate frontend container.

• Use Docker Compose to build, start, inspect, test, and stop all services.

• Persist database records using a named Docker volume.

3. Application overview

The application predicts an Iris flower species from four numeric measurements. The browser displays the predicted species and confidence. Each successful prediction is saved in MongoDB and can be viewed in prediction history.

Service	Technology	Purpose	Port
Frontend	Nginx + HTML/CSS/JS	Displays page and sends API requests	8080 → 80
AI Model	Python, Flask, Scikit-learn	Predicts Iris species	5000 → 5000
Database	MongoDB 7.0	Stores prediction history	27017 internal

Networking: The AI container connects to MongoDB using the Compose service hostname `mongodb`. Browser JavaScript calls `http://localhost:5000` because the browser runs on the host machine, outside the Docker network.

4. Prerequisites and Docker verification

• Windows with WSL 2 and Ubuntu installed.

• Docker Desktop installed and running.

• Docker Desktop → Settings → Resources → WSL Integration → enable the Ubuntu distribution.

• Internet connection for downloading images and Python packages.

4.1 Open Ubuntu and verify Docker

```
docker --version
docker compose version
docker info
```

Version commands should print Docker and Compose versions. docker info should show server information.

4.2 Fix permission error (only if required)

```
sudo usermod -aG docker $USER
newgrp docker
docker run hello-world
```

If permission is still denied, close and reopen Ubuntu and try again. You may temporarily use sudo before Docker commands. Ensure Docker Desktop is running and WSL integration is enabled.

5. Create the project folders

```
cd ~
mkdir -p docker-compose-ai/ai-model
mkdir -p docker-compose-ai/frontend
cd ~/docker-compose-ai
pwd
```

Expected project structure:

```
docker-compose-ai/
■■■ docker-compose.yml
■■■ ai-model/
```

```
■   ■■■ app.py
■   ■■■ requirements.txt
■   ■■■ Dockerfile
■■■ frontend/
    ■■■ index.html
    ■■■ Dockerfile
```

Use nano to create each file. Example: nano ai-model/app.py. Paste the complete code, save with Ctrl+O, press Enter, and exit with Ctrl+X. Repeat for every file.

6. Create the AI Model service

6.1 Create ai-model/app.py

```
nano ai-model/app.py
```

```
from flask import Flask, request, jsonify
from flask_cors import CORS
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from pymongo import MongoClient
from datetime import datetime
import os
import time

app = Flask(__name__)
CORS(app)

MONGO_URL = os.getenv("MONGO_URL", "mongodb://mongodb:27017/")
client = MongoClient(MONGO_URL, serverSelectionTimeoutMS=5000)
db = client["iris_database"]
collection = db["predictions"]

# Load Iris data and train the Random Forest model
iris = load_iris()
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(iris.data, iris.target)

@app.route("/", methods=["GET"])
def home():
    return jsonify({
        "message": "AI Model API is running",
        "status": "success"
    })

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json(silent=True)
    if not isinstance(data, dict):
        return jsonify({"error": "Invalid JSON"}), 400

    features = data.get("features")
    if not isinstance(features, list) or len(features) != 4:
        return jsonify({"error": "Enter exactly 4 features"}), 400

    try:
        features = [float(value) for value in features]
        prediction = int(model.predict([features])[0])
        probabilities = model.predict_proba([features])[0]

        result = {
            "prediction": str(iris.target_names[prediction]),
            "class_id": prediction,
            "confidence": round(float(max(probabilities)), 4)
        }

        collection.insert_one({
            "features": features,
            "prediction": result["prediction"],
            "confidence": result["confidence"],
            "timestamp": datetime.utcnow()
        })
        return jsonify(result)

    except (ValueError, TypeError):
        return jsonify({"error": "Features must be numeric"}), 400
    except Exception:
        return jsonify({"error": "Prediction or database error"}), 500

@app.route("/history", methods=["GET"])
def history():
    records = list(
        collection.find({}, {"_id": 0})
        .sort("timestamp", -1)
        .limit(20)
    )
    for record in records:
```

```
        record["timestamp"] = record["timestamp"].isoformat()
    return jsonify(records)

if __name__ == "__main__":
    # Wait for MongoDB to become available
    for attempt in range(15):
        try:
            client.admin.command("ping")
            print("Connected to MongoDB successfully")
            break
        except Exception:
            print("Waiting for MongoDB...")
            time.sleep(2)
    else:
        raise RuntimeError("Could not connect to MongoDB")

    app.run(host="0.0.0.0", port=5000)
```

Explanation: Flask provides the API. Scikit-learn loads the Iris dataset and trains a Random Forest classifier. POST /predict accepts four measurements, predicts a species, and saves the result. GET /history returns the latest 20 saved predictions. The startup loop waits for MongoDB to respond.

6.2 Create ai-model/requirements.txt

```
nano ai-model/requirements.txt
```

```
Flask
flask-cors
scikit-learn
pymongo
```

Flask runs the API; flask-cors allows browser requests; scikit-learn supplies the model; pymongo connects Python to MongoDB.

6.3 Create ai-model/Dockerfile

```
nano ai-model/Dockerfile
```

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --timeout=300 --retries=10 -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

FROM selects the base image; WORKDIR sets the working directory; COPY adds files; RUN installs packages; EXPOSE documents the API port; CMD starts the Flask program.

7. Create the Frontend service

7.1 Create frontend/index.html

```
nano frontend/index.html
```

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Iris Flower Prediction</title>
<style>
* { box-sizing: border-box; }
body { margin:0; font-family:Arial,sans-serif; background:#eef2f7; color:#172033; }
.container { max-width:760px; margin:45px auto; padding:28px; background:white;
  border-radius:14px; box-shadow:0 8px 28px rgba(0,0,0,.08); }
h1 { text-align:center; color:#17365d; }
.intro { text-align:center; color:#5b6575; }
.grid { display:grid; grid-template-columns:repeat(2,1fr); gap:15px; margin-top:25px; }
label { display:block; font-weight:bold; margin-bottom:6px; }
input { width:100%; padding:12px; border:1px solid #cbd5e1; border-radius:7px; font-size:16px; }
button { border:0; border-radius:7px; padding:12px 18px; margin:18px 8px 0 0;
  background:#1d4ed8; color:white; cursor:pointer; font-size:15px; }
button.secondary { background:#475569; }
#result,#history { margin-top:20px; padding:15px; background:#f8fafc;
  border:1px solid #e2e8f0; border-radius:8px; }
.record { padding:9px 0; border-bottom:1px solid #e2e8f0; }
@media(max-width:560px) { .container{margin:15px;padding:18px;} .grid{grid-template-columns:1fr;} }
</style>
</head>
<body>
<main class="container">
<h1>Iris Flower Prediction</h1>
<p class="intro">Enter four flower measurements to predict its species.</p>
<div class="grid">
  <div><label for="sepalLength">Sepal length (cm)</label>
  <input id="sepalLength" type="number" step="any" value="5.1"></div>
```

```
  <div><label for="sepalWidth">Sepal width (cm)</label>
  <input id="sepalWidth" type="number" step="any" value="3.5"></div>
  <div><label for="petalLength">Petal length (cm)</label>
  <input id="petalLength" type="number" step="any" value="1.4"></div>
  <div><label for="petalWidth">Petal width (cm)</label>
  <input id="petalWidth" type="number" step="any" value="0.2"></div>
</div>
<button onclick="predictFlower()">Predict Flower</button>
<button class="secondary" onclick="loadHistory()">View History</button>
<div id="result">Prediction result will appear here.</div>
<div id="history"></div>
</main>
<script>
const API_URL = "http://localhost:5000";

async function predictFlower() {
  const values = ["sepalLength","sepalWidth","petalLength","petalWidth"]
    .map(id => Number(document.getElementById(id).value));

  if (values.some(value => !Number.isFinite(value))) {
    document.getElementById("result").textContent =
      "Please enter valid numeric measurements.";
    return;
  }

  const resultBox = document.getElementById("result");
  resultBox.textContent = "Predicting...";
  try {
    const response = await fetch(API_URL + "/predict", {
      method: "POST",
      headers: {"Content-Type":"application/json"},
      body: JSON.stringify({features:values})
    });
    const data = await response.json();
    if (!response.ok) throw new Error(data.error || "Prediction failed");

    resultBox.textContent = "Predicted species: " + data.prediction +
      " | Confidence: " + (data.confidence * 100).toFixed(2) + "%";
    loadHistory();
  } catch (error) {
    resultBox.textContent = "Error: " + error.message;
  }
}

async function loadHistory() {
  const historyBox = document.getElementById("history");
  historyBox.textContent = "Loading history...";
  try {
    const response = await fetch(API_URL + "/history");
    if (!response.ok) throw new Error("Could not load history");
    const records = await response.json();
    historyBox.replaceChildren();

    const heading = document.createElement("h3");
    heading.textContent = "Recent Prediction History";
    historyBox.appendChild(heading);

    if (records.length === 0) {
      const empty = document.createElement("p");
      empty.textContent = "No predictions saved yet.";
      historyBox.appendChild(empty);
      return;
    }

    records.forEach(record => {
      const item = document.createElement("div");
      item.className = "record";
      item.textContent = record.prediction + " | Confidence: " +
        (record.confidence * 100).toFixed(2) + "% | " + record.timestamp;
      historyBox.appendChild(item);
    });
  } catch (error) {
    historyBox.textContent = "Error: " + error.message;
  }
}
</script>
</body>
</html>
```

The page contains four measurement inputs, a Predict button, and a View History button. The browser calls Flask; it never connects directly to MongoDB.

7.2 Create frontend/Dockerfile

```
nano frontend/Dockerfile
```

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

Nginx serves the static HTML page on port 80 inside the frontend container.

8. Create docker-compose.yml

```
nano docker-compose.yml
```

```
services:
  ai-model:
    build: ./ai-model
    container_name: iris-ai-model
    ports:
      - "5000:5000"
    environment:
      - MONGO_URL=mongodb://mongodb:27017/
    depends_on:
      mongodb:
        condition: service_healthy
    restart: unless-stopped

  mongodb:
    image: mongo:7.0
    container_name: iris-mongodb
    volumes:
      - mongo-data:/data/db
    healthcheck:
      test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping')"]
      interval: 10s
      timeout: 5s
      retries: 5
    restart: unless-stopped

  frontend:
    build: ./frontend
    container_name: iris-frontend
    ports:
      - "8080:80"
    depends_on:
      - ai-model
    restart: unless-stopped

volumes:
  mongo-data:
```

This file defines all services. The AI image is built from ai-model; MongoDB uses the official mongo:7.0 image; the frontend is built from frontend. MongoDB's health check ensures it is ready before the AI API starts. The named volume preserves database files.

8.1 Compose settings explained

• services: components/containers in the application.

• build: directory containing the Dockerfile to build an image.

• image: image to download and run, here MongoDB.

• ports: host-to-container port mapping; 8080:80 means host port 8080 forwards to container port 80.

• environment: environment variables passed into a container.

• depends_on: defines service startup dependency.

• healthcheck: tests whether MongoDB is responding.

• volumes: persistent storage mounted at /data/db.

• restart: unless-stopped: automatically restarts a service unless it was manually stopped.

9. Verify files and validate YAML

```
cd ~/docker-compose-ai
find . -maxdepth 3 -type f -print
docker compose config
```

The find command should list the six project files. docker compose config checks the YAML and displays the resolved configuration. YAML indentation is important; use spaces, not tabs.

10. Build and start all services

```
docker compose up --build -d
```

This command downloads the MongoDB image if needed, builds the AI and frontend images, creates the Compose network and named volume, and starts the services in the background. The AI service waits for MongoDB's health check. The first build can

take several minutes.

If the Docker socket permission error appears, use this form instead:

```
sudo docker compose up --build -d
```

Wait for the command to finish. Do not close Docker Desktop while testing.

11. Check containers and logs

```
# Show Compose services and status
docker compose ps

# Show all running containers
docker ps

# Display logs for every service
docker compose logs

# Follow live logs (Ctrl+C stops log viewing, not containers)
docker compose logs -f

# Display one service's logs
docker compose logs ai-model
docker compose logs mongodb
docker compose logs frontend
```

You should see three services: iris-ai-model, iris-mongodb, and iris-frontend. MongoDB should report healthy. The AI and frontend services should be running.

12. Test the application

12.1 Open the frontend

Open Chrome or another browser in Windows and visit:

```
http://localhost:8080
```

The Iris Flower Prediction page should appear.

12.2 Test the AI health endpoint

```
curl http://localhost:5000/
```

Expected JSON contains a message that the AI Model API is running and status is success.

12.3 Send a prediction request

```
curl -X POST http://localhost:5000/predict \
  -H "Content-Type: application/json" \
  -d '{"features":[5.1,3.5,1.4,0.2]}'
```

The response contains prediction, class_id, and confidence. The precise confidence value is generated by the model.

12.4 Read prediction history

```
curl http://localhost:5000/history
```

After one or more predictions, this endpoint should return saved records as a JSON array.

12.5 Test in the browser

• Keep the default four measurements or enter your own numeric values.

• Click Predict Flower.

• Check the predicted species and confidence.

• Click View History to display saved prediction records.

13. Verify records in MongoDB

```
docker exec -it iris-mongodb mongosh
```

At the MongoDB prompt, execute:

```
show dbs
use iris_database
show collections
db.predictions.find().pretty()
exit
```

The predictions collection contains the four input features, predicted class, confidence, and timestamp for each successful prediction.

14. Stop, start, and remove services

14.1 Stop services (keep containers)

```
docker compose stop
```

14.2 Start stopped services

```
docker compose start
```

14.3 Remove containers and network

```
docker compose down
```

docker compose down removes the Compose containers and network but normally preserves the named mongo-data volume. The saved database records remain.

14.4 Delete the database volume (optional and destructive)

```
docker compose down -v
```

WARNING: -v removes the named volume and permanently deletes the saved MongoDB data. Do not use this if you want to keep prediction history.

15. Troubleshooting

Problem	Solution
Docker daemon / permission denied	Start Docker Desktop; enable Ubuntu WSL integration. Apply the docker group commands in Section 4.2 or temporarily use sudo.
Port 5000 already allocated	Check docker ps. Stop the older Practical 9 container if appropriate: docker stop iris-ai-container.
Port 8080 already allocated	Stop the conflicting process or change Compose mapping to 8081:80, then browse to http://localhost:8081.
AI cannot connect to MongoDB	Run docker compose ps and docker compose logs mongodb ai-model. Check MONGO_URL is mongodb://mongodb:27017/ and M
Frontend loads but prediction fails	Check curl http://localhost:5000/ and docker compose logs ai-model. Confirm port 5000 is published.
Package download timeout	Retry docker compose up --build -d. The Dockerfile uses pip timeout and retry options. Check internet connection.
YAML indentation error	Open nano docker-compose.yml and correct spaces/indentation. Run docker compose config before building.
Code changes are not visible	Rebuild and recreate: docker compose up --build -d.

16. End-to-end working

• User enters four measurements in the web page served by Nginx.

• Browser JavaScript sends POST /predict as JSON to Flask at localhost:5000.

• Flask validates the input and converts the four values to numbers.

• The Random Forest model predicts one of the three Iris species and calculates the highest class probability.

• Flask inserts the input, prediction, confidence, and timestamp into MongoDB.

• Flask returns JSON; the browser displays the predicted species and confidence.

• For history, the browser calls GET /history and Flask retrieves the latest records from MongoDB.

Architecture flow: Browser → Nginx Frontend → Flask AI API → Scikit-learn model. Flask also communicates with MongoDB to save and retrieve prediction history.

17. Important concepts for the practical

Image vs container

An image is a packaged template. A container is a running instance created from an image.

Dockerfile vs Compose file

A Dockerfile describes how to build one image. docker-compose.yml describes how multiple services work together, including ports, dependencies, environment variables, and volumes.

Port mapping

8080:80 maps host port 8080 to container port 80. 5000:5000 publishes the Flask API on host port 5000.

Service-name DNS

Compose creates a network for the services. Containers reach each other by service name, so the AI API connects to hostname mongodb rather than localhost.

Volume

A named volume stores database files independently of a container's writable layer. Here mongo-data is mounted at MongoDB's /data/db.

Health check and depends_on

The dependency condition delays the AI service until MongoDB reports healthy. The Python startup retry loop performs an additional database connection check.

18. Result

Docker Compose was used to orchestrate a web-based AI application through three separate services: an Nginx frontend, a Flask and Scikit-learn AI API, and a MongoDB database. The application was tested through the browser and API endpoints, and prediction history was stored using a persistent named volume.

19. Viva questions and answers

1. What is Docker Compose?
A tool for defining and running multi-container Docker applications using a YAML file.

2. Why is Docker Compose used here?
To build, configure, start, and manage the frontend, AI API, and database together.

3. What is a service?
A containerized component defined in the Compose file with its image/build, ports, environment, and dependencies.

4. What is a Dockerfile?
A text file containing instructions to build a Docker image.

5. Why use three containers?
Each service has a separate responsibility and can be managed independently.

6. Why use MongoDB?
To store prediction history as documents.

7. What is a volume?
Persistent storage that can outlive a container.

8. Why does the AI service use hostname mongodb?
Compose provides service-name DNS on its network.

9. Why does browser JavaScript use localhost?
The browser runs on the host and reaches the published API port through localhost:5000.

10. What does 8080:80 mean?
Host port 8080 forwards to port 80 inside the frontend container.

11. What does docker compose up --build -d do?
Builds images, creates resources, starts services, and runs them in detached mode.

12. What does docker compose down do?
Stops and removes Compose containers and network; named volumes are normally preserved.

13. What is a health check?
A periodic test to determine whether a service is responding correctly.

14. What is Random Forest?
An ensemble algorithm that combines predictions from multiple decision trees.

15. What is the Iris dataset?
A standard dataset containing measurements of three Iris flower species.

16. What is an API endpoint?
A URL path and HTTP method used by a client, such as POST /predict.

17. What is JSON?
A text format used to exchange structured data between applications.

18. What is Nginx used for?
It serves the static frontend page to the browser.

19. Does the browser connect directly to MongoDB?
No. It calls Flask; Flask performs the database operations using PyMongo.

20. How do you check running services?
Use docker compose ps or docker ps.

20. Final practical checklist

• Docker and Compose versions verified.

• Project folder and all six files created.

• AI model and frontend images built.

• MongoDB container started and healthy.

• All three services visible in docker compose ps.

• Frontend opened at http://localhost:8080.

• AI health endpoint tested at http://localhost:5000/.

• Prediction tested in browser or with curl.

• History verified through /history and MongoDB shell.

• Services stopped safely without deleting the database volume.

Suggested screenshots for the journal: project directory/file structure, successful docker compose ps, frontend page, prediction result, prediction history, and MongoDB collection output.